

Reliability of FINN-Generated CNN Accelerators for Image Classification on SRAM-Based FPGAs Under Heavy-Ion-Induced Faults

Fabio Benevenuti¹, Arthur F. Ely¹, Nilberto H. Medina¹, Nemitala Added¹, Vitor Ângelo P. Aguiar¹, Eduardo L. A. Macchione¹, Saulo G. Alberton¹, Greiciane J. Cesário¹, Matheus S. Pereira¹, Marcilei A. Guazzelli¹, Antonio Carlos S. Beck¹, José Rodrigo Azambuja¹, and Fernanda L. Kastensmidt¹

Abstract—This study examines the performance of two convolutional neural networks (CNNs) designed for aerial image classification in the presence of radiation-induced bit-flips. We modify these CNNs by adjusting parameters such as quantization and parallelism to facilitate their implementation using the FINN inference engine, which is optimized for the AMD/Xilinx field programmable gate arrays (FPGAs). The aim is to evaluate the impact of different quantization levels, network topologies, and architectural parallelism on area, computational performance, and reliability in the presence of soft-errors. Emulated fault injection and heavy ion irradiation were performed. The results indicate that the same CNN topology can exhibit up to a 2.7× difference in mean fluence to failure (MΦTF) by altering quantization and architectural parallelism. The findings demonstrate that higher dependability can be obtained by carefully combining a suitable CNN topology with optimized quantization and architectural parallelism.

Index Terms—Artificial neural networks, field-programmable gate arrays (FPGAs), high-level synthesis (HLS), image classification, radiation effects, reliability, single-event effects (SEEs).

I. INTRODUCTION

CONVOLUTIONAL neural networks (CNNs) have demonstrated strong performance in image classification tasks [1]. By utilizing large datasets of labeled images for training, CNNs can achieve high accuracy in identifying land cover types, natural and artificial structures, vegetation, and other features commonly found in aerial imagery applications [2]. A significant bottleneck for space applications is the limited bandwidth and availability of satellite downstream for data transfer. This issue arises due to the growing number of satellites competing for radio channels and the relatively small number of Earth-based stations [3]. Consequently, in-orbit data processing has become highly valuable for space applications. Besides classification, CNNs can also perform image segmentation and object detection. This approach allows designers to reduce the amount of data transmitted from satellites, providing users with faster and more refined results compared to processing raw data on Earth. This method can even achieve near-real-time computation, which is crucial for applications like early flood and wildfire warnings. Integrating CNNs with in-orbit satellite imagery classification is a logical progression.

However, optimized hardware to process CNNs requires specific computing performance, resources, accuracy, and reliability according to the different application requirements. This set of requirements is an open challenge for satellite systems, as they have strictly constrained resources and conflicting targets in size, weight, power, and cost (SWaP-C), and their components can be susceptible to radiation-induced soft errors. Additionally, the *new space* [4] move toward commercial grade electronics demands fault tolerance techniques to cope with the radiation-rich environment. In this context, using volatile static memory cells (SRAM)-based field programmable gate arrays (FPGAs) to implement embedded CNNs offers significant advantages. FPGAs are also combined with microprocessors in the all-programmable system on chip (APSoC) devices, thus allowing programmability through both software and dedicated programmable logic. The reconfigurability of such devices enables exploration of tradeoffs between

Received 28 March 2025; revised 17 June 2025; accepted 7 July 2025. Date of publication 14 July 2025; date of current version 18 August 2025. This work was supported in part by the Office of Naval Research Global (ONR) under Grant N629092212014; in part by the Coordination for the Improvement of Higher Education Personnel, Brazil (CAPES)-Finance Code 001; in part by the Council for Technological and Scientific Development, Brazil (CNPq); in part by the UNIESPAÇO Program under Grant 407302/2022-0; in part by Universal CNPq under Grant 404054/2023-4; in part by the Program for the Support of Technological Development of the Semiconductor and Display Industry, Brazil (PADIS) through HT Micron Semicondutores; in part by the Competence Center for Digital Agriculture signed with Brazilian Company of Research and Industrial Innovation (EMBRAPII) under Grant 056/2023; in part by the Rio Grande do Sul Research Foundation, Brazil (FAPERGS); and in part by the São Paulo Research Foundation, Brazil (FAPESP) under Grant 2023/16053-8. (Corresponding author: Fernanda L. Kastensmidt.)

Fabio Benevenuti and Arthur F. Ely are with the Instituto de Informatica, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre 91501-970, Brazil (e-mail: fbenevenuti@inf.ufrgs.br; afely@inf.ufrgs.br).

Nilberto H. Medina, Nemitala Added, Vitor Ângelo P. Aguiar, Eduardo L. A. Macchione, Saulo G. Alberton, Greiciane J. Cesário, and Matheus S. Pereira are with the Instituto de Física, Universidade de São Paulo (USP), São Paulo 05508-090, Brazil (e-mail: medina@if.usp.br; nemitala@if.usp.br; vpaguiar@if.usp.br; macchion@if.usp.br; alberton@if.usp.br; jcesario@if.usp.br; mpereira@if.usp.br).

Marcilei A. Guazzelli is with the Departamento de Física, Centro Universitário da FEI, São Bernardo do Campo 09850-901, Brazil (e-mail: marcilei@fei.edu.br).

Antonio Carlos S. Beck, José Rodrigo Azambuja, and Fernanda L. Kastensmidt are with the Instituto de Informatica, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre 91501-970, Brazil, and also with the Centro de Competência em Agricultura Digital, SENAI-RS, Embrapii, São Leopoldo 93025-753, Brazil (e-mail: caco@inf.ufrgs.br; jose.azambuja@inf.ufrgs.br; fglima@inf.ufrgs.br).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TNS.2025.3588691>.

Digital Object Identifier 10.1109/TNS.2025.3588691

CNN quantization and throughput and system costs (e.g., power, energy, and resource usage) while also supporting the integration of fault-tolerant mechanisms to improve system reliability. The development starts by defining the target accuracy of the CNN, as this is typically a critical factor for the applications. Then, an embedded inference engine can be optimized to minimize system costs while maintaining the required accuracy level.

One of the key parameters in this optimization process is quantization, which involves altering the precision of the operands, such as weights, and simplifying arithmetic operations, for instance, adjusting values to the nearest power-of-two and replacing multipliers by shift operations. Quantization can vary from low- to high-bit-width numeric formats and has a direct impact on computational resources, caches, local and external memories, and interconnection infrastructure. Another crucial parameter is the level of parallelism in the CNN architecture, which directly affects the system's throughput. The next challenge is to assess the susceptibility of CNNs to soft errors. Radiation-induced faults in APSoCs can have a lasting effect on the configuration memory bits, which can be mitigated by redundancy and corrected through reconfiguration or memory scrubbing. Therefore, it is essential to analyze CNNs under fault injection to identify the most vulnerable parts of the design, enabling decision-making regarding architecture and implementation of effective mitigation techniques, and qualify the CNNs under radiation. As FPGA technology advances, CNNs are expected to be executed within embedded systems on satellite boards while adhering to mission constraints. There are different approaches to implementing CNNs in embedded systems: 1) using hardware accelerators described in register-transfer level (RTL) [5], [6] or a higher-level, such as C++, to be later synthesized by high-level synthesis (HLS) tools, such as hls4ml [7], FINN [8], and ZynqNet [9]; or 2) in software that will execute on specialized hardware, such as deep-learning processor unit (DPU) [10] and GPUs [11]. The performance of the neural network may vary according to the inference engine implementation and its parameters.

In this work, we explore two CNNs designed to classify aerial images using the FINN engine [8] running on a Xilinx Zynq-7000 FPGA. To the best of our knowledge, there is no formal expansion for the name FINN, but it loosely stands for *fast neural network inference*. FINN was initially designed for binarized (1 bit) neural networks and is currently also used with other numeric representations. It includes an end-to-end flow to integrate CNNs in APSoCs, capable of delivering high throughput for in-orbit imaging applications. For this reason, we chose FINN as our case-study inference engine. We explore the design space for CNNs by modifying quantization and parallelism levels. The Zynq APSoC features a processor system based on an Arm Cortex-A9 CPU acting as a host processor combined with an FPGA fabric, where FINN is implemented as an out-of-core CNN accelerator. Radiation-induced bit-flips were emulated by fault injection in the bitstream [12] to analyze the reliability of different implementations of the FINN engine accelerator. The results can guide designers in selecting the best quantization and parallelism to achieve the highest reliability. Selected CNN implementations are then tested under heavy ion irradiation.

II. FINN ENGINE FLOW

The FINN engine is a framework developed by AMD/Xilinx for designing and deploying efficient neural networks on FPGAs. It is particularly well-suited for CNNs because it focuses on accelerating deep learning inference on programmable hardware. FINN emphasizes quantized neural networks (QNNs), where weights and activations are represented with reduced precision, down to 1 bit. QNNs tackle network size and power consumption by compressing full-precision parameters (i.e., weights, activations, and biases) into significantly smaller representations. This optimization substantially reduces memory requirements, resource usage, and processing power, making QNNs highly appealing for deployment on resource-constrained embedded devices.

The FINN workflow consists of three primary steps: training, building, and deployment. In the training phase, PyTorch, a Python library for deep learning [13], and Brevitas, a PyTorch-based library for neural network quantization [14], are employed. During this phase, quantization for each layer of the CNN is specified. A basic approach to quantization is post-training quantization (PTQ), where the bit-width of parameters is reduced after training. Although this method is straightforward, it can lead to significant accuracy loss in complex CNN models. To mitigate this, Brevitas implements quantization-aware training (QAT), where the model is trained with reduced bit-width parameters from the start, minimizing accuracy loss but requiring more computational resources to train. In the build step, FINN uses the trained QNN model, saved in open neural network exchange (ONNX) format [15], to create a hardware CNN accelerator. During this phase, the ONNX model is optimized and transformed into HLS layers, which are then converted into Verilog code. FINN outputs a Vivado project containing the Stitched intellectual property core (IP), ready for implementation in an FPGA board.

While generating custom FPGA implementations of QNNs, FINN allows for parameterization to optimize performance throughput by increasing the parallelism of the CNN layers. This is achieved by adjusting the folding factor, a build parameter that determines how much CNN modules implemented in the FPGA are reused by multiplexing in time. In FINN, folding is inversely proportional to performance and resource utilization: increasing the folding factor reduces parallelism, resulting in smaller hardware with lower resource consumption.

Parallelization can be configured in two ways. One method involves providing detailed FINN's builder configuration in a text file specifying parameters such as the number of processing elements (PEs) and single-instruction multiple-data (SIMD) operations for each layer of the model. Alternatively, a target throughput can be passed as a parameter, allowing the framework to determine the folding configuration automatically. For this project, we opt for the latter approach to explore and utilize this feature of the FINN framework.

III. RADIATION-INDUCED SOFT-ERRORS

SRAM-based FPGAs are particularly susceptible to radiation because they store their configuration data in volatile static memory cells (SRAMs). Unlike nonvolatile FPGA technologies (e.g., flash-based or antifuse-based FPGAs), SRAM-based

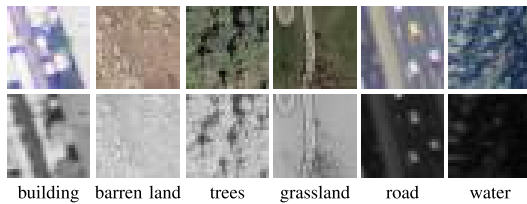


Fig. 1. Sample RGB and NIR SAT-6 images [16].

devices must reload their configuration upon power-up and are continuously vulnerable to radiation-induced errors.

In current technology, the main type of radiation-induced effect in SRAM-based FPGAs is the single event effect (SEE). These faults occur when high-energy particles, such as protons, neutrons, or heavy ions, strike sensitive regions of the FPGA and can lead to errors or malfunctions. Single event upsets (SEUs) occur when radiation particles alter the state of a memory element, such as a bit-flip in an SRAM cell or flip-flop (FF). In SRAM-based FPGAs, SEUs can affect both end-user data flowing through the circuit and the application circuit itself. An SEU in the FPGA's configuration memory (CRAM) can alter combinational logic, the behavior of storage elements, clock and signal routing through the FPGA fabric, and other critical aspects of the circuit. Due to the significantly larger volume of CRAM compared to end-user data, SEUs in CRAM are of primary concern.

Moreover, other radiation-induced faults can occur in SRAM-based FPGAs. Single event transients (SETs) happen when a radiation-induced effect causes a transient voltage pulse in the logic. Single event functional interrupt (SEFI) occurs when an SEU causes the design in the FPGA to become unresponsive. They may involve the corruption of critical control registers or even the complete loss of configuration data, requiring a reconfiguration or reset of the FPGA.

When analyzing a design without fault tolerance in SRAM-based FPGAs, its susceptibility usually depends on its architecture, its ability to mask faults, and its area. The area can be approximated by the number of essential bits. Essential bits in SRAM-based FPGAs refer to the specific configuration bits that, when altered or corrupted (e.g., due to radiation-induced faults), can directly impact the functionality of the design. These bits control the critical aspects of the FPGA's logic, interconnects, and memory, and any errors can lead to functional failures in the implemented circuit.

IV. CONFIGURATION MEMORY SCRUBBING

Many SRAM-based FPGAs have built-in memory scrubbing capabilities. Memory scrubbing in SRAM-based FPGAs is a fault tolerance technique used to correct CRAM faults caused by SEUs, which can lead to incorrect logic behavior, malfunction, or system failure. Scrubbing ensures that the FPGA operates reliably over time by continuously checking and correcting errors in the configuration memory. Configuration scrubbing involves periodically reading the contents of the configuration memory and comparing them against a known correct copy or data redundancy, such as error detection and correction codes (ECC/EDAC). If a bit-flip is detected, the scrubber automatically corrects the corrupted bit by rewriting the configuration data back into the FPGA's CRAM.

While the memory scrubbing can perform corrective actions autonomously, the SEU correction by scrubbing is not instantaneous. Consequently, the application circuit in the FPGA must cope with eventual minor faults until scrubbing takes action. The main benefit of memory scrubbing is to prevent the SEUs from accumulating and leading to more significant issues. In this work, we explore many parameterizations and optimizations in FINN combined with scrubbing to achieve high levels of reliability and accuracy. This also reflects how well the end-to-end computation approach of FINN can mask the impact of occasional SEUs throughout the CNN computation, providing some leeway for the scrubber's actions.

V. RELATED WORK

CNNs for image classification using commercial-grade SRAM-based FPGAs onboard satellites must present low power due to energy limitations, high throughput, and low area to minimize the occurrence of faults. However, the majority of studies have focused on running CNNs on the ground. Basu et al. [16] and Liu et al. [17] implemented land-cover classification into four and six classes, with methods including CNNs, using computers and GPUs on the ground. Manning et al. [18] also utilized CNNs at the ground to classify images captured onboard the International Space Station (ISS). A notable work for in-orbit classification is [19], with a neural network for land-cover classification of images in four categories, using a microprocessor and a dedicated neural co-processor. Other applications of CNNs in orbit include data and quality management as described in [20], [21], and [22]. A common characteristic of these CNNs is the small number of image classes to be handled. A small number of classes is also found at the CNN model for the CIFAR 10 dataset selected to be included in the GPU for Space (GPU4S) benchmark funded by the European Space Agency (ESA) [23].

Few works have evaluated the FINN engine under radiation-induced faults. Souvatzoglou et al. [24] analyzed a quantized version of the FINN framework for the MNIST dataset under bit-flip fault injection in an FPGA, focusing on varying the folding parameter in the PEs. Khoshavi et al. [25] also injected faults into neural networks running on a Zynq-7000 APSoC, and in [26], a customized version of the FINN engine was adapted to emulate data faults. Their objective was to identify which parts of the CNN were most susceptible to inference misprediction. The reliability of CNNs under radiation has also been studied in [27], which examined networks implemented using hls4ml and Vitis HLS [28], analyzing failure rates under neutron irradiation and how the parallelism of the generated accelerator affects its reliability. However, it does not explore how quantization or changes in topology influence this characteristic. Similarly, Benevenuti et al. [29] and Libano et al. [30] investigated neutron-induced faults and fault injection in CNN accelerators. Benevenuti et al. [29] used the ZynqNet inference engine with neural network topologies similar to the presented in this work (Section VI-B), but did not explore how changes in the number of layers, quantization, or parallelism in the generated CNN accelerator affect reliability. While Libano et al. [30] examined quantization, it does not address the other two parameters. Furthermore, Zhao et al. [31] evaluated SEUs

in CNNs implemented on FPGAs under fault injection and alpha particle radiation. However, like [29], it does not investigate how different CNN configurations impact accelerator reliability.

In contrast to these works, our study provides a more comprehensive evaluation of the chosen CNN and accelerator engine under fault injection and radiation-induced scenarios. Unlike prior studies that analyzed individual aspects, such as quantization or parallelism, we systematically explore the combined impact of quantization, folding, and CNN topology on the reliability of accelerators. By testing variations in the number of layers in the CNN topology, our research highlights how these changes influence the resilience of CNN inference under fault injection and heavy-ion radiation. This multifaceted approach, alongside the use of the FINN framework on Zynq-7000 APSoC, distinguishes our work and provides new insights into accelerator reliability. To the best of our knowledge, no prior work has systematically addressed this combination of factors. Furthermore, no published paper has analyzed the FINN engine on Zynq FPGAs under fault injection and radiation scenarios.

VI. PROPOSED DESIGN METHODOLOGY

We propose a design space exploration to investigate the behavior of CNNs implemented in the FINN engine on SRAM-based FPGAs running under radiation-induced faults. After selecting a case-study application, we design and train CNNs capable of achieving high accuracy in the target task. The framework allows exploration of quantization and folding configurations, resulting in different implementations with varying performance and area characteristics. Additionally, FPGA synthesis parameters, such as using block RAM (BRAM) or distributed RAM (LUTRAM), can be adjusted. Each design may exhibit different levels of susceptibility to faults based on its architecture and resource usage.

A. Case-Study Aerospace Application

The case-study application uses the SAT-6 dataset [16], with six distinct classes for land cover, as illustrated in Fig. 1. The dataset is sourced from the United States National Agriculture Imagery Program (NAIP) and covers the continental part of the United States and is composed of four-band images in red, green, and blue (RGB), and near infrared (NIR) bands. We used the dataset with its original organization with 324 000 images for training and 81 000 images for testing. The number of samples in each class, for both the training and the test subsets, follows roughly the proportion: building: 5%, barren land: 23%, trees: 18%, grassland: 16%, roads: 3%, and water: 37%.

B. CNN Topology

The two CNNs featured in this study are depicted in Fig. 2, namely topology 1 (t1), with two convolutional layers, and topology 2 (t2), with four convolutional layers. These CNN models, acquired through a genetic algorithm seen in [32] that generated optimized topologies for the SAT-6 dataset, achieved approximately 98% accuracy, yet this accuracy level is contingent upon the specific inference engine utilized.

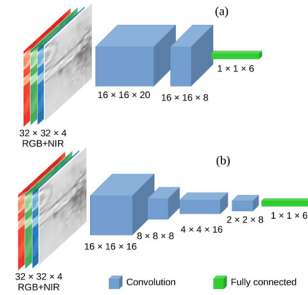


Fig. 2. Case-study CNN topologies. (a) Topology 1 (t1). (b) Topology 2 (t2).

We chose these two topologies because they have a different number of layers, thus allowing us to better evaluate how aggressive quantization impacts each topology's accuracy.

C. Quantization

The CNNs were trained from scratch in PyTorch, using the Brevitas library for quantization. A new training process was started for each quantization configuration in 8, 4, and 2 bits, identified in the following as w8, w4, and w2, respectively. For training, we adopted a learning rate of 3×10^{-4} , a batch size of 512 images, and 30 training epochs. These parameters were adjusted iteratively, and this setting was chosen because it produced the best results for most configurations. The 2 bits quantization was discarded after the training process due to its low accuracy.

D. Folding

In terms of folding, a higher desired throughput results in less folding, leading to greater parallelization. Three target throughput levels—500, 5000, and 50 000 frames per second (FPS)—were selected to test the automatic folding feature of FINN. All these combinations produced nine distinct implementations of the FINN accelerator. The throughput level of 500 FPS, for instance, implies that the accelerator should classify 500 images per second.

The effective throughput in FPS varied depending on the topology and due to the framework's automatic folding, resulting in satisfactory, though not entirely accurate, throughput. For a target of 50 000 FPS, the functionality minimized folding, leading to higher hardware parallelization and resource utilization, despite not reaching the requested throughput. Fig. 3 shows an example of folding level variation for topology 1.

Besides the hardware folding, the CNN processing throughput is also limited by the amount of logical and storage resources in the FPGA. The AMD Xilinx Zynq-7000 APSoC model XC7Z020 was selected for the experiments. The implementation of FINN is illustrated in Fig. 4. The FINN's main communication interface is the Arm AMBA Advanced eXtensible Interface Stream (AXI-S). A test controller was implemented to feed FINN with test images.

E. FINN Test Controller

A specialized IP was developed to control the FINN engine, which can be customized based on the CNN architecture

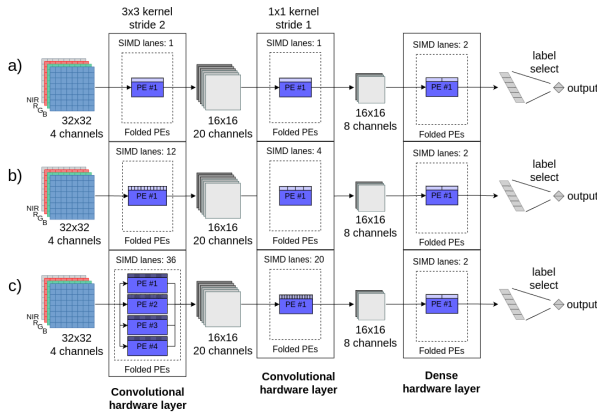


Fig. 3. Folding configuration of the CNN layers when targeting (a) 500, (b) 5000, and (c) 50000 FPS.

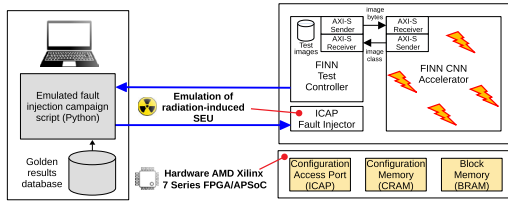


Fig. 4. FINN test setup for irradiation and fault injection.

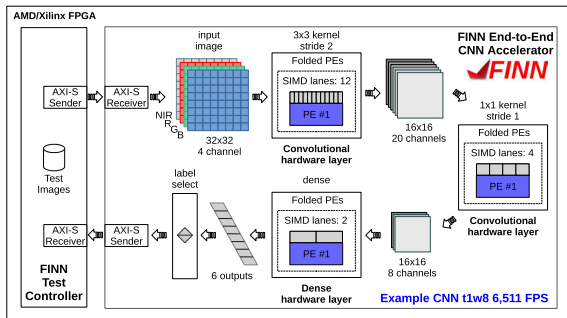


Fig. 5. FINN test controller and an FINN accelerator.

implemented by the accelerator, as well as its quantization and folding parameters. The FINN test controller handles the transmission of image bytes and retrieves the inference results after the neural network processing. Test images are stored in BRAMs within the IP, and it sends one byte per cycle to the accelerator. Communication is carried out via AXI-S. Fig. 5 shows our case-study FINN accelerator implementing one test case of CNNs: topology 1 with 8 bits of quantization and folding to achieve 6511 FPS (t1w8 6511 FPS).

F. Exploring the BRAM and LUTRAM Storage

The two primary types of memory available inside most FPGAs for storing CNN weights and bias constants are BRAM and LUTRAM. Each has distinct characteristics that influence performance, resource utilization, power consumption, and reliability. BRAM consists of dedicated memory blocks specifically designed for large data storage and offers more storage capacity compared to LUTRAM. LUTRAM repurposes the lookup tables (LUTs) in the FPGA's configurable logic blocks (CLBs) to act as small memory elements. Since LUTs are primarily designed for logic functions, LUTRAM usually provides less storage capacity compared to BRAM. LUTRAM

can be implemented as small, distributed memory elements, making it more suited for storing small amounts of data or smaller bit-wise weights in neural networks. In this work, we implemented various FINN engines, some implementations storing the CNN weights in the BRAM and others implementing them in the LUTRAM.

Table I presents the design information of the two case-studied CNNs topologies: t1 and t2. In Table I, we analyze the CNN topology t1 with 8 bits of quantization and four different performances, and the CNN topology t2 with 4 and 8 bits of quantization and four different performances. Results in terms of accuracy and resource utilization (FFs, LUTs, LUTRAM, BRAM, and essential bits) are presented. Note that by increasing the required performance (reducing the folding of the PEs), the resource utilization increases significantly. CNN topology t1 with 8 bits of quantization presents the highest resource utilization numbers when running at the highest performance.

VII. EMULATED FAULT INJECTION METHODOLOGY

The fault injection methodology emulates radiation-induced SEUs by accumulating bit-flips in the FPGA's CRAM [12]. Both fault position and injection time are randomized. The fault rate is adjusted so that no more than one fault is injected per CNN processing cycle. The fault injection tool uses Xilinx's internal configuration access port (ICAP) hardware block to manipulate the contents of the CRAM. A small Fault Injector module (see Fig. 4) is synthesized together with the CNN, but isolated from it, using floorplan constraints, to prevent fault injection from disrupting the injector itself. The faults injected affect the configuration of BRAM, DSP, CLB, and routing paths. A test campaign script executing in the control computer communicates with the Fault Injector module, informing the position in CRAM where a fault must be injected. The Test Controller (see Section VI-E) was implemented with a small set of test images to continuously stimulate the CNN during tests. For simplicity, this set of images includes only images correctly classified by the CNN. The expected label for each test image is known by the test campaign script. If the CNN classifies a test image with the wrong label or fails to provide the classification result in the allotted time, then it is recorded as a functional failure, together with the number of accumulated faults injected until the failure. Upon failure, the FPGA is reprogrammed, and the process is repeated until the desired number of failure events is recorded.

VIII. FAULT INJECTION RESULTS

The number of accumulated faults injected (n), recorded for each observed failure, allows the creation of histograms and the cumulative probability distribution functions for failure [$F(n)$]. The reliability curves can be computed as the reciprocal of failure [$R(n) = 1 - F(n)$]. These curves are presented in Figs. 6 and 7. One can observe that a large number of essential bits and area resource usage are not directly correlated to a low reliability of the FINN accelerator. The FINN implementation that presents the lowest reliability is topology 2 with 8 bits of quantization with the target throughput of 10847 FPS with

TABLE I

ANALYSIS OF ACCURACY, RESOURCES UTILIZATION, # OF ESSENTIAL BITS, MFTF (FAULT INJECTION), AND MΦTF (HEAVY IONS) FOR DIFFERENT IMPLEMENTATIONS OF THE FINN ACCELERATOR

CNN topology	Accuracy (%)	Constants storage	Throughput (FPS)	# of FFs	# of LUTs	# of LUTRAM	# of BRAM (36Kib)	# of essential bits	Fault injection			Heavy ions					
									Faults injected	Failures observed	MFTF (faults)	Scrub. off			Scrub. on		
												Fluence (p/cm ²)	Failures	MΦTF (p/cm ²)	Fluence (p/cm ²)	Failures	MΦTF (p/cm ²)
t2w8	98.80%	BRAM	10,847 FPS	19,527	42,940	1,890	6.0	6.2×10^6	74,898	1,001	75	849×10^3	107	7.9×10^3	880×10^3	107	8.2×10^3
		LUTRAM	41,501 FPS	20,362	44,374	2,096	0.0	6.4×10^6	39,276	501	78	549×10^3	105	5.2×10^3	868×10^3	107	8.1×10^3
		BRAM	5,427 FPS	10,154	26,227	1,144	9.0	3.6×10^6	123,372	1,001	123	858×10^3	99	8.7×10^3	1146×10^3	104	11.0×10^3
		LUTRAM	5,427 FPS	10,257	26,852	1,048	6.5	3.7×10^6	62,071	501	124	451×10^3	50	9.0×10^3	906×10^3	104	8.7×10^3
		BRAM	679 FPS	9,279	25,354	1,112	6.5	3.5×10^6	137,739	1,001	138	1245×10^3	106	11.7×10^3	1469×10^3	111	13.2×10^3
		LUTRAM	679 FPS	9,232	25,979	1,016	4.5	3.5×10^6	63,064	501	126	1218×10^3	114	10.7×10^3	1273×10^3	106	12.0×10^3
t2w4	96.70%	BRAM	10,851 FPS	11,603	12,193	1,252	4.0	2.3×10^6	96,806	1,006	96	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
		LUTRAM	41,501 FPS	11,886	12,507	1,320	0.0	2.3×10^6	47,660	501	95	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
		BRAM	5,428 FPS	6,333	7,792	774	8.0	1.5×10^6	132,474	1,001	132	780×10^3	101	7.7×10^3	1357×10^3	104	13.0×10^3
		LUTRAM	5,428 FPS	6,377	8,018	726	6.0	1.5×10^6	62,931	501	126	1453×10^3	104	14.0×10^3	n.t.	n.t.	n.t.
		BRAM	679 FPS	5,697	7,476	758	6.5	1.4×10^6	144,898	1,001	145	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
		LUTRAM	679 FPS	5,659	7,707	710	4.5	1.4×10^6	71,358	501	142	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
t1w8	98.58%	BRAM	16,271 FPS	26,801	45,496	604	4.5	7.7×10^6	116,596	1,001	116	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
		LUTRAM	41,501 FPS	27,357	46,695	860	0.0	8.1×10^6	49,943	101	100	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
		BRAM	6,511 FPS	20,598	18,386	404	11.5	4.1×10^6	230,522	1,001	230	1749×10^3	101	17.3×10^3	2406×10^3	101	23.8×10^3
		LUTRAM	6,511 FPS	20,883	20,116	404	5.5	4.4×10^6	103,344	501	206	1639×10^3	109	15.0×10^3	2074×10^3	119	17.4×10^3
		BRAM	543 FPS	19,515	17,028	420	7.5	3.9×10^6	303,932	1,001	304	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.
		LUTRAM	543 FPS	19,604	18,767	420	2.5	4.2×10^6	130,930	501	261	n.t.	n.t.	n.t.	n.t.	n.t.	n.t.

n.t.: not tested

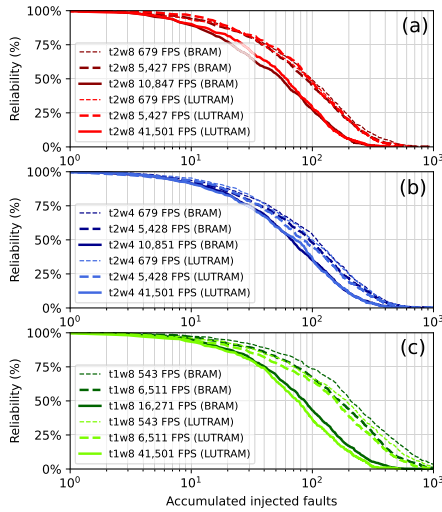


Fig. 6. Reliability curves under emulated fault injection on CRAM: CNN topology 2 with (a) 8 and (b) 4 bits quantization, CNN topology 1 with (c) 8 bits quantization.

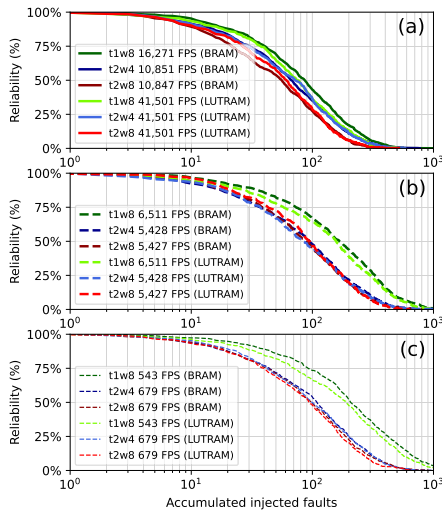


Fig. 7. Reliability curves under fault injection. (a) Folding for maximum throughput. (b) 5000 FPS. (c) 500 FPS.

2 673 789 essential bits, while the one that presents the highest reliability is topology 1 with 8 bits of quantization with the target throughput of 543 FPS with 4 242 203 essential bits.

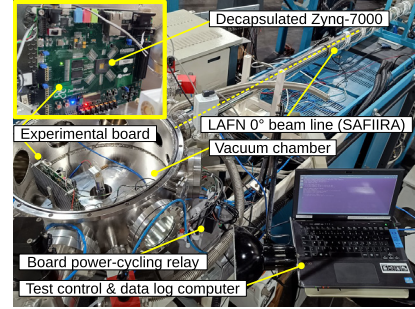


Fig. 8. Experimental board setup at the SAFIIRA facility at the LAFN 0° beamline.

The second-best implementation in terms of reliability is still topology 1 with 8 bits of quantization and a higher target throughput. It seems that overall, the CNN topology 1 with different folding is always better than CNN topology 2, except when the area resources and essential bits get too high. These results are significant as they highlight the importance of training the CNN with an appropriate and optimized topology and quantization. Although CNN topology 2 achieved high accuracy with lower bit quantization and reduced area resource usage, it was less resilient to radiation-induced bit-flips when compared to other topologies.

The ratio between injected faults and number of observed failures (N) gives the mean faults to failure ($MFTF = \Sigma n/N$), as shown in Table I. CNN topology 1 also presents the highest number of MFTFs.

IX. HEAVY-ION TEST EXPERIMENTAL SETUP

The same CNN hardware and test controller used in fault injection were implemented for radiation tests. Also, the same set of test images and the same policies for failure detection and recording were adopted. Irradiation tests were performed at the SAFIIRA facility [33] at the 0° beamline of the Pelletron 8UD tandem accelerator housed at the Nuclear Physics Open Laboratory (LAFN) at the University of São Paulo (USP), Brazil. Fig. 8 shows the board being installed inside the experimental chamber. The SAFIIRA facility was designed specifically for tests on electronic devices and allows irradiation at low flux with beam energy uniformity better than

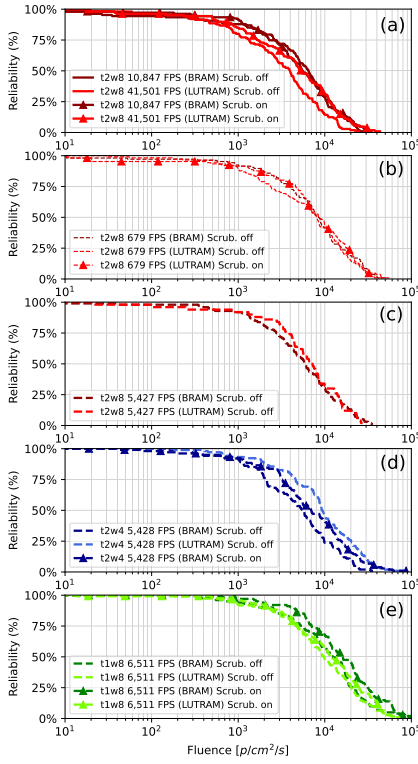


Fig. 9. Reliability curves under heavy ions irradiation: CNN topology 2 with 8 bits quantization and (a) folding for maximum throughput, (b) for 500 and (c) 5000 FPS, (d) CNN topology 2 with 4 bits and folding for 5000 FPS, and (e) CNN topology 1 with 8 bits and folding for 5000 FPS.

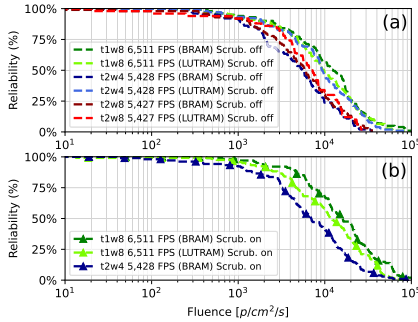


Fig. 10. Reliability curves under heavy ions irradiation: comparison with CRAM memory scrubbing (a) OFF and (b) ON and folding for target 5000 FPS.

90%. The tests were performed in vacuum with a ^{16}O beam and an energy of 40.8 MeV on the surface of the device. Beam penetration of 29.5 μm in silicon was estimated with the SRIM [34] software package, with linear energy transfer (LET) at the device surface of 5.2 MeV/mg/cm² in silicon.

The average particle flux ($\bar{\phi}$) was measured for each test run with an uncertainty of 16%. The particle flux is continuously monitored and integrated from a photodiode mounted inside the irradiation chamber. Flux measurement is calibrated at least twice a day with reference to a surface barrier detector placed in the center of the irradiation chamber, in front of the device under test. Flux measurements were also periodically cross-checked by performing static tests in the FPGA by readback of the CRAM and BRAM contents. During the experiments, the average particle flux for test runs ranged from 4×10^2 to 2×10^3 p/cm²/s.

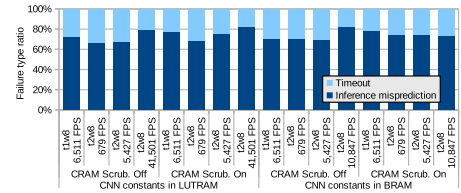


Fig. 11. Failure diagnosis for applications under radiation.

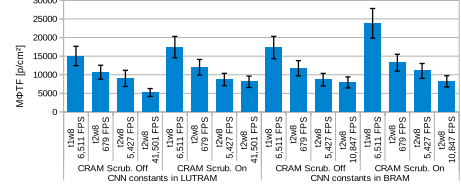


Fig. 12. MΦTF for CNN constants in BRAM and LUTRAM. Error bars represent 1σ .

X. HEAVY-ION IRRADIATION RESULTS

The number of functional failures observed for each CNN configuration tested under radiation is indicated in Table I. The mean fluence to failure (MΦTF) is computed as the ratio between the test fluence (Φ) and number of observed failures (N) ($M\Phi TF = \Phi/N$), while the fluence for a given test run was obtained from the total time of the run multiplied by the average flux ($\bar{\phi}$) of the run ($\Phi = \bar{\phi} \times \Sigma t$). For comparison, the MΦTF is presented in Table I aside from its analogous MFTF from fault injection. The reliability curves obtained from observed failure events under radiation are presented in Fig. 9. For configurations with CRAM scrubbing disabled, it can be compared with reliability curves from Fig. 6. For convenience, the reliability curves for the designs with folding for a target throughput of 5000 FPS are grouped in Fig. 10, in configurations with and without CRAM scrubbing. The results from Fig. 10(a) can also be compared with Fig. 7(b).

The functional failures can be observed by the test instrumentation either as an inference misprediction, when the image class reported by the CNN mismatches the known class of the input image, or as a timer run-out (timeout), when the CNN does not respond in the expected time frame. The proportion of these two failure types is presented in Fig. 9. The strong presence of inference mispredictions suggests the use of ECCs and modular redundancy to mitigate radiation effects and improve CNN reliability. As the FINN inference engine design is generated by HLS from code in C++ language, and it uses intensive matrix and matrix-vector multiplication, mitigation techniques can be implemented at that high-level specification as demonstrated in [35] and [36].

From Table I, we observed an increased footprint in essential bits, which has a potential impact on reliability, when moving CNN constant weights from BRAM to LUTRAM. The increase ranged from 1% to 7% in the number of essential bits. Radiation result shows that CRAM memory scrubbing did not compensate for this loss in reliability. It is also summarized in Fig. 12, with MΦTF for configurations storing CNN constants in BRAM and LUTRAM, with and without memory scrubbing.

It is noteworthy, however, that in the case of FINN folding for maximum throughput, the use of LUTRAM allowed for

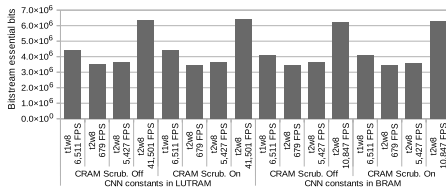
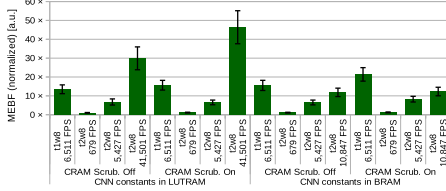


Fig. 13. Bitstream essential bits in FPGA configuration memory.

Fig. 14. MEBF, normalized with respect to t2w8 679 FPS. Error bars represent 1σ .

an increase in the CNN throughput in $3\times$ for CNN topology t1 and $3.8\times$ for topology t2. For the topology t2 at maximum throughput, tested under radiation, the loss in reliability was less than 2% in $M\Phi TF$ when using LUTRAM with scrubbing active.

Also comparing results from Table I and Fig. 13, we observe that, in many cases, reliability metrics degrade while increasing the number of essential bits, which represents the area of SRAM cells in the configuration memory of the design on FPGAs. This larger area is also associated with higher throughput, less folding and higher parallelism, and neural network complexity in terms of number of CNN layers, number of parameters for weights and bias, or number of multiply-accumulate operations.

Despite the potential increase in area, higher throughput of the CNN can compensate for the effects of radiation, delivering more computation before a functional failure. The requirement for high throughput also comes from potential applications onboard satellites. For instance, a satellite image with ground resolution of 16.5 m per pixel covering an area of 95×95 km, at a ground speed near 7 km/s, would produce one non-overlapping image approximately every 14 s. If a CNN with an input of 32×32 pixels is used to build a semantic map of non-overlapping images, it would require a CNN throughput over 2500 FPS for quasi-real-time processing. Higher throughput is required with overlapping pixels or if images are stored for processing only when full power is available from solar panels.

This can also be put in perspective using the metric for mean execution between failure (MEBF $\simeq M\Phi TF \times FPS$), which is presented in the chart of Fig. 14. Despite having the worst $M\Phi TF$ (see Fig. 12), the CNNs with higher throughput can deliver higher MEBF (see Fig. 14). Moreover, this type of application with higher intensive computation shows the benefits of combining the storage of CNN constants in LUTRAM with the healing provided by memory scrubbing.

XI. MAGNITUDE OF COMPUTATIONAL ERRORS

The last layer of the CNN performs the label select operation, as shown in Figs. 3 and 5. It was modified to further investigate the computation errors in the CNN probability outputs associated with the inference misprediction in Fig. 11,

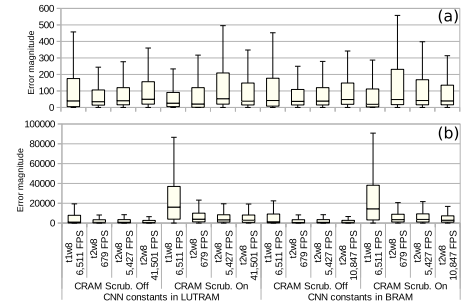


Fig. 15. Error magnitude at output probability: (a) masked errors and (b) errors translated to an inference misprediction.

as well as the masked errors. The change consisted of replacing the label select operation in hardware with its equivalent in software. This change exposed the output of the fully connected layer (see Fig. 2) to the test fixture. There is also variation in the number of essential bits (see Table I) due to the changes in the label select and test instrumentation to capture the probability outputs. Output errors under radiation-induced faults were evaluated using emulated fault injection, as shown in Section VII. The same failure detection and classification criteria were adopted in this analysis. The same fault rate and fault density per bit were adopted for all CNN implementations.

During this test, it was performed 8.7×10^6 inference cycles in the 16 CNN configurations in Fig. 11, out of which only 0.09% resulted in image class misprediction, hence a functional failure. Meanwhile, another 6% of the inference cycles resulted in an error in the probability output in the last CNN layer but did not change the ranking of the highest probability. These errors are not functional failures because they did not translate into CNN inference misprediction. Moreover, they are not externally observable in the normal use of the CNN accelerator with label select in hardware, as depicted in Figs. 3 and 5. However, these masked errors may still affect other functions that support the CNN, such as the computation of inference confidence, if present.

Fig. 15 presents the difference between the known output probability for the test images and the output probability under emulated fault injection. The output probability in the CNN accelerator for 8 bits quantization is a signed 24-bit integer value. The chart presents absolute values of the error. The range of error is significantly narrower in the case of masked errors [see Fig. 15(a)] when compared to errors leading to inference misprediction [see Fig. 15(b)].

XII. COMPARISON OF FAULT INJECTION VERSUS HEAVY-IONS

When comparing the results from fault injection and radiation, one can see that the ranking order in the reliability metrics is consistent for most CNN configurations. Deviations are noticeable when comparing configurations using BRAM and LUTRAM to store CNN constants because the contents of BRAM are not reachable by fault injection during the CNN processing, leading to optimistic reliability metrics for designs using BRAM. Notwithstanding, emulated fault injection provided useful information, at a very low cost, during design

space exploration, screening FINN design strategies, and selection of candidate implementations for irradiation tests.

XIII. CONCLUSION

This study demonstrates that the FINN engine offers significant flexibility in the project design phase to achieve the requirements of area, performance, power consumption, and reliability. This adaptability is crucial for the space industry, where satellites encounter diverse environments and missions, ranging from intensive data processing, which demands high performance, to energy-efficient operations, which require low resource usage. Results have shown that CNN topology 1 with 8-bit quantization implemented in the FINN accelerator is more reliable than the other versions, despite having more essential bits. This highlights the importance of selecting the right CNN topology for reliability. Both results from fault injection and radiation have presented the same trend when exercising different topologies, quantization, and folding.

REFERENCES

- [1] L. Chen, S. Li, Q. Bai, J. Yang, S. Jiang, and Y. Miao, "Review of image classification algorithms based on convolutional neural networks," *Remote Sens.*, vol. 13, no. 22, Nov. 2021, Art. no. 4712.
- [2] J. Liu, T. Wang, A. Skidmore, Y. Sun, P. Jia, and K. Zhang, "Integrated 1D, 2D, and 3D CNNs enable robust and efficient land cover classification from hyperspectral imagery," *Remote Sens.*, vol. 15, no. 19, Oct. 2023, Art. no. 4797.
- [3] M. G. C. Branco and A. da Rocha Gomes, "Satellite communication challenges in a fully interconnected world," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Aug. 2017, pp. 624–628.
- [4] D. Paikowsky, "What is new space? The changing ecosystem of global space activity," *New Space*, vol. 5, no. 2, pp. 84–88, Jun. 2017.
- [5] I. C. Lopes, F. Benevenuti, F. L. Kastensmidt, A. A. Susin, and P. Rech, "Reliability analysis on case-study traffic sign convolutional neural network on APSoc," in *Proc. IEEE 19th Latin-American Test Symp. (LATS)*, Mar. 2018, pp. 7–12.
- [6] K. Abdelouahab, M. Pelcat, J. Sérot, C. Bourrasset, and F. Berry, "Tactics to directly map CNN graphs on embedded FPGAs," *IEEE Embedded Syst. Lett.*, vol. 9, no. 4, pp. 113–116, Dec. 2017.
- [7] T. Aarrestad et al., "Fast convolutional neural networks on FPGAs with hls4ml," *Mach. Learn., Sci. Technol.*, vol. 2, no. 4, Jul. 2021, Art. no. 045015.
- [8] Y. Umuroglu et al., "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2017, pp. 64–74.
- [9] H.-B. Wang, Y.-S. Wang, J.-H. Xiao, S.-L. Wang, and T.-J. Liang, "Impact of single-event upsets on convolutional neural networks in Xilinx Zynq FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 68, no. 4, pp. 394–401, Apr. 2021.
- [10] P. Maillard et al., "Radiation-tolerant deep learning processor unit (DPU)-based platform using Xilinx 20-nm kintex UltraScale FPGA," *IEEE Trans. Nucl. Sci.*, vol. 70, no. 4, pp. 714–721, Apr. 2023.
- [11] H. G. M. Hernandez, "Towards the efficient multi-platform execution of deep neural networks," in *Proc. 31st Int. Conf. Field-Program. Log. Appl. (FPL)*, Aug. 2021, pp. 277–278.
- [12] F. Benevenuti and F. L. Kastensmidt, "Comparing exhaustive and random fault injection methods for configuration memory on SRAM-based FPGAs," in *Proc. IEEE Latin Amer. Test Symp. (LATS)*, Mar. 2019, pp. 87–92.
- [13] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Conf. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [14] A. Pappalardo and G. Franco, (Feb. 2024). *Xilinx/Brevitas*. Accessed: Aug. 25, 2024. [Online]. Available: <https://zenodo.org/records/10680356>
- [15] The Linux Foundation. (Aug. 2024). *Open Neural Network Exchange*. Accessed: Aug. 25, 2024. [Online]. Available: <https://onnx.ai/>
- [16] S. Basu, S. Ganguly, S. Mukhopadhyay, R. DiBiano, M. Karki, and R. Nemani, "DeepSat: A learning framework for satellite imagery," in *Proc. 23rd SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2015, Art. no. 37.
- [17] Q. Liu et al., "DeepSat v2: Feature augmented convolutional neural nets for satellite image classification," *Remote Sens. Lett.*, vol. 11, no. 2, pp. 156–165, Feb. 2020.
- [18] J. Manning et al., "Machine-learning space applications on SmallSat platforms with TensorFlow," *Proc. 32nd AIAA/USU Conf. Small Satell.*, Jul. 2018, Paper 458.
- [19] R. Velazco, P. Cheynet, A. Tissot, J. Haussy, J. Lambert, and R. Ecoffet, "Evidences of SEU tolerance for digital implementations of artificial neural networks: One year MPTB flight results," in *Proc. 5th Eur. Conf. Radiat. Effects Compon. Syst. (RADECS)*, Sep. 1999, pp. 565–568.
- [20] I. van Zyl Marais, W. H. Steyn, and J. A. du Preez, "Construction of an image quality assessment model for use on board an leo satellite," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2008, pp. II-1068–II-1071.
- [21] E. El-Araby, T. El-Ghazawi, J. Le Moigne, and R. Irish, "Reconfigurable processing for satellite on-board automatic cloud cover assessment," *J. Real-Time Image Process.*, vol. 4, no. 3, pp. 245–259, Jan. 2009.
- [22] A. Popowicz, "Image processing in the BRITE nano-satellite mission," *Proc. SPIE*, vol. 9904, Jul. 2016, Art. no. 99041R.
- [23] L. Kosmidis et al., "GPU4S: Embedded GPUs in space—latest project updates," *Microprocessors Microsyst.*, vol. 77, Sep. 2020, Art. no. 103143.
- [24] I. Souvatzoglou, D. Agiakatsikas, A. Sari, V. Vlagkoulis, A. Tavoularis, and M. Psarakis, "Tradeoff between performance and reliability in FPGA accelerated DNNs for space applications," in *Proc. Eur. Data Handling Data Process. Conf. (EDHPC)*, Oct. 2023, pp. 390–396.
- [25] N. Khoshavi, C. Broyles, and Y. Bi, "Compression or corruption? A study on the effects of transient faults on BNN inference accelerators," in *Proc. 21st Int. Symp. Quality Electron. Design (ISQED)*, Mar. 2020, pp. 99–104.
- [26] N. Khoshavi, C. Broyles, Y. Bi, and A. Roohi, "Fiji-FIN: A fault injection framework on quantized neural network inference accelerator," in *Proc. 19th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2020, pp. 1139–1144.
- [27] M. Traiola, F. F. dos Santos, P. Rech, C. Cazzaniga, O. Sentieys, and A. Kritikakou, "Impact of high-level synthesis on reliability of artificial neural network hardware accelerators," *IEEE Trans. Nucl. Sci.*, vol. 71, no. 4, pp. 845–853, Apr. 2024.
- [28] Advanced Micro Devices, Inc. (Nov. 2024). *Vitis HLS*. Accessed: Nov. 20, 2024. [Online]. Available: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html>
- [29] F. Benevenuti et al., "Neutron-induced faults on CNN for aerial image classification on SRAM-based FPGA using softcore GPU and HLS," in *Proc. 21st Eur. Conf. Radiat. Effects Compon. Syst. (RADECS)*, Sep. 2021, pp. 57–60.
- [30] F. Libano, B. Wilson, M. Wirthlin, P. Rech, and J. Brunhaver, "Understanding the impact of quantization, accuracy, and radiation on the reliability of convolutional neural networks on FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 67, no. 7, pp. 1478–1484, Jul. 2020.
- [31] X. Zhao et al., "Single event effects evaluation on convolution neural network in Xilinx 28 nm system on chip," *Chin. Phys. B*, vol. 33, no. 7, Jun. 2024, Art. no. 078501.
- [32] F. Benevenuti et al., "Investigating the reliability impacts of neutron-induced soft errors in aerial image classification CNNs implemented in a softcore SRAM-based FPGA GPU," *Microelectron. Rel.*, vol. 138, Nov. 2022, Art. no. 114738.
- [33] V. A. P. Aguiar et al., "SAFIIRA: A heavy-ion multi-purpose irradiation facility in Brazil," *Rev. Sci. Instrum.*, vol. 91, no. 5, pp. 053301-1–053301-9, May 2020.
- [34] J. F. Ziegler, M. D. Ziegler, and J. P. Biersack, "SRIM—The stopping and range of ions in matter (2010)," *Nucl. Instrum. Methods Phys. Res. Sect. B: Beam Interact. With Mater. At.*, vol. 268, nos. 11–12, pp. 1818–1823, Jun. 2010.
- [35] A. F. d. Santos, L. A. Tambara, F. Benevenuti, J. Tonfat, and F. L. Kastensmidt, "Applying TMR in hardware accelerators generated by high-level synthesis design flow for mitigating multiple bit upsets in SRAM-based FPGAs," in *Proc. Int. Symp. Appl. Reconfigurable Comput.*, 2017, pp. 202–213.
- [36] G. S. Rodrigues, J. Fonseca, F. Benevenuti, F. Kastensmidt, and A. Bosio, "Exploiting approximate computing for low-cost fault tolerant architectures," in *Proc. 32nd Symp. Integr. Circuits Syst. Design (SBCCI)*, ACM, Aug. 2019, Art. no. 3.